

Tutorial

Java File I/O

Mark G. Eramian

University of Saskatchewan

Opening a File

- In Java, files are abstracted by creating an instance of the `java.io.File` class.
- It requires the filename as the parameter.
- Creating an `File` object does not open the file.

```
1 import java.io.File;
2
3 public class SomeClass {
4     public static void main(String args[]) {
5         /* Creates a file object */
6         File myFile = File('filename.txt');
7     }
8 }
```

Reading Text Files

- Text files can be opened and read using the `java.util.Scanner` class.
- An instance of `Scanner` can be created from an instance of `File`.

```
1  import java.io.File;
2  import java.util.Scanner;
3
4  public class SomeClass {
5      public static void main(String args[]) {
6          /* Create a file object *and*
7             open it for reading. */
8          File myFile = File('filename.txt');
9          Scanner infile = new Scanner(myFile);
10     }
11 }
```

Reading Text Files

- The file is not actually “opened” until you instantiate Scanner so you should use a try-catch block to make sure the file was opened successfully. For example:

```
1 import java.io.File;
2 import java.io.Scanner;
3 import java.io.FileNotFoundException;
4
5 public class SomeClass {
6     public static void main(String args[]) {
7         Scanner infile = null;
8         try {
9             infile = new Scanner(new File('filename.txt'));
10        }
11        catch (FileNotFoundException e) {
12            System.out.println("Error: file not found.");
13            return;
14        }
15    }
16 }
```

Using Scanner

- *Tokens* are sequences of characters from text files that are separated by delimiters.
- `Scanner` can be used to read the next token from an open file.
- The default delimiters are whitespace.
- `Scanner` has methods to read the next token and convert it to a particular data type.
- Example: the `Scanner.nextInt()` method reads the next token from the file and tries to convert it to an integer, throwing an exception if that is not possible.
- `Scanner` also has methods to test whether the next token in the file can be interpreted as a particular data type, e.g. `hasNextInt()`.

Example: Read a List File of Integers

- Recall from CMPT 141: a list file is a file with one data item (token) per line.
- The code below reads a file with one integer per line and adds them up.

```
1 public static void main(String args[]) {
2     Scanner infile = null;
3     try {
4         infile = new Scanner(new File('filename.txt'));
5     }
6     catch (FileNotFoundException e) {
7         System.out.println("Error: file not found.");
8         return;
9     }
10    int sum = 0;
11    while( infile.hasNextInt() ) {
12        sum = sum + infile.nextInt();
13    }
14    infile.close()
15 }
```

Error Checking

- Previous example works as long as every line has a valid number.
- If a non-integer is encountered, the file reading stops and later valid integers are not read.
- We can use the `hasNext()` method, which returns true as long as there is another token to be read regardless of its interpretation, to distinguish between reaching the end of the file and reaching an invalid token.
- This lets us continue adding integers in the remainder of the file even if we encounter a non-integer at some point.

Example: Read a List File with Better Error Checking

```
1 public static void main(String args[]) {
2     Scanner infile = null;
3     try {
4         infile = new Scanner(new File('filename.txt'));
5     }
6     catch (FileNotFoundException e) {
7         System.out.println("Error: file not found.");
8         return;
9     }
10    int sum = 0;
11    // As long as there is another token (of any type)...
12    while( infile.hasNext() ) {
13        try {
14            // ... try to read it as an integer and add it to the running sum.
15            sum = sum + infile.nextInt();
16        }
17        catch( InputMismatchException e ) {
18            // If the token read was not an integer, issue a warning.
19            System.out.println("Warning: non-integer token.");
20        }
21        // Since we caught the exception, we can *continue* reading
22        // more data even if there is an error on one line.
23    }
24    infile.close()
25 }
```

Other Useful Scanner Methods

- `hasNextFloat()`, `nextFloat()`: check for/read float tokens
- `hasNextLine()`, `nextLine()`: check for/read entire lines (without tokenizing). Useful for reading human-readable text, or when you want to read a line and tokenize it yourself.
- `next()`: return the next token, whatever it is (this is how you read string-type tokens).
- `useDelimiter(pattern)`: sets the delimiter to exactly `pattern` if you want to use a delimiter other than spaces.
- Full listing of `Scanner` methods in the Java documentation.

Reading Other File Formats

- Recall from CMPT 141 *tabular files*: fixed number of data items per line separated by delimiters; data items on one line may be of different types, but i -th data item on a line is always the same type.
- Two approaches in Java:
 - Write logic to read each line, reading each token with appropriate `Scanner.nextInt()`, `Scanner.nextFloat()`.
 - Use `Scanner.nextLine()` to read each line, use `String.trim()` and `String.split()` in much the same way as `strip()` and `split()` are used in python.
 - In the latter approach, all tokens will be strings, and you have to make the appropriate type conversion yourself after splitting.

Exercise 1

- Write java code to read a file where each line contains (in order):
 - Month (string)
 - Year (string)
 - Min temp (float)
 - Max temp (float)
- Items on each line are separated by whitespace.
- Store the data as a `Java.util.ArrayList` of `TempRecord` objects (which just stores the four data items and has getters/setters).

Writing Files

- There are a mind-wrenching number of ways to write data to a file in Java.
- We'll use the `BufferedWriter` class. The ex

```
1 import java.io.FileWriter;
2 import java.io.IOException;
3 import java.io.BufferedWriter;
4
5 public class WriteFileExample {
6     public static void main(String args[]) {
7         // Open a file for writing
8         BufferedWriter outfile = null;
9         try {
10             outfile = new BufferedWriter(new FileWriter("filename.txt"));
11         }
12         catch (IOException e) {
13             System.out.println("Error: file cannot be opened.");
14             return;
15         }
16     }
17 }
```

Opening Files for Writing

- There are a mind-wrenching number of ways to write data to a file in Java.
- We'll use the `BufferedWriter` class. The example below shows how to open a file for writing.

```
1  import java.io.FileWriter;
2  import java.io.IOException;
3  import java.io.BufferedWriter;
4
5  public class WriteFileExample {
6
7      public static void main(String args[]) {
8          // Open the file
9
10         BufferedWriter outfile = null;
11         try {
12             outfile = new BufferedWriter(new FileWriter("stuff.txt"));
13         }
14         catch (IOException e) {
15             System.out.println("Error: file cannot be opened.");
16             return;
17         }
18     }
19 }
```

Writing data to Files

- The `BufferedWriter.write(s)` method will write the string `s` to the file opened for writer.
- Only strings may be written. Other data types must be converted before being passed to `BufferedWriter.write()`.
- All desired whitespace, including newlines, must be explicitly written.
- All `BufferedWriter.write()` calls have to be in a try-catch block because the method can throw the checked exception `IOException`.
- You must close the file with the `BufferedWriter.close()` method (also must be in try-catch block) or not all of the data you wrote will actually make it to disk.

Exercise 2

Write code to write the values of the following variables to a file:

```
1 String s = "The answer to the ultimate question of life,"+
2           "the universe, and everything is: ";
3 int fortyTwo = 42;
4 float fortyTwoPointZero = 42.0;
```

Make sure that each variable's value is on a separate line.

Concluding Remarks

- File I/O in java is much uglier than in Python.
- There are other ways to do file I/O beyond what is shown here. This is just the basics.
- What you've seen here is enough to get you through CMPT 280.
- Online tutorials on Java File I/O are numerous and, in the course instructor's opinion, universally awful.